Carnegie Mellon University Africa

Assignment Report

Conditioning a Sensor Telemetry System Using Python Programming and Linear Algebra

Name: Jean Claude IRADUKUNDA
Email: cleveri@andrew.cmu.edu
Program: M.S. Electrical and Computer Engineering

Task

Develop a Python-based telemetry conditioning system that applies structured programming techniques and Linear Algebra concepts to process noisy sensor data stored in a JSON file. The final application reads the telemetry stream, removes invalid measurements, computes summary statistics, detects peaks and reports meaningful engineering information.

Aim

The objective of this assignment was to reinforce Python programming skills while applying Linear Algebra concepts through a practical engineering application. The work focused on building a modular telemetry processing pipeline capable of reading, cleaning, analysing and summarising noisy sensor measurements.

Methodology

The solution was developed using a modular problem decomposition approach where each processing stage was implemented as an independent Python function. The telemetry pipeline begins by reading a JSON file using the `open()` function before converting the JSON object into Python objects using `json.load()`. Since the loaded data is represented as dictionaries, it was transformed into a list of tuples to simplify iteration and numerical processing.

The telemetry stream was then inspected to identify missing readings (`None`) and values outside the valid operating range of $[0, 100]$. Invalid samples were removed before computing statistical measures such as the total sum and average sensor reading. Peak detection was subsequently performed on the cleaned data, after which only significant peaks with readings greater than 22 were retained. Finally, the processed results were summarised by reporting the number of valid samples, average reading, detected peaks and the maximum time interval between significant peaks.

Function Summary

Table 1: Summary of the implemented functions

Function	Purpose
my_sum(y_data)	Computes and returns the total sum of valid sensor readings.
keep_valid(streamed_data)	Filters out missing (`None`) and out-of-range values, returning only valid telemetry samples.
get_average()	Calculates the average value of the valid sensor readings.
find_peaks(valid)	Detects local peaks from the conditioned telemetry stream.
load_stream(path)	Reads the JSON telemetry file, converts it into Python objects and transforms dictionaries into a list of tuples.
filtered_peaks(peaks)	Selects significant peaks whose reading values are greater than 22.

Results

The developed telemetry conditioning pipeline successfully processed the noisy dataset by loading the JSON telemetry file, removing invalid measurements and computing the required statistical information. The execution output is presented below.

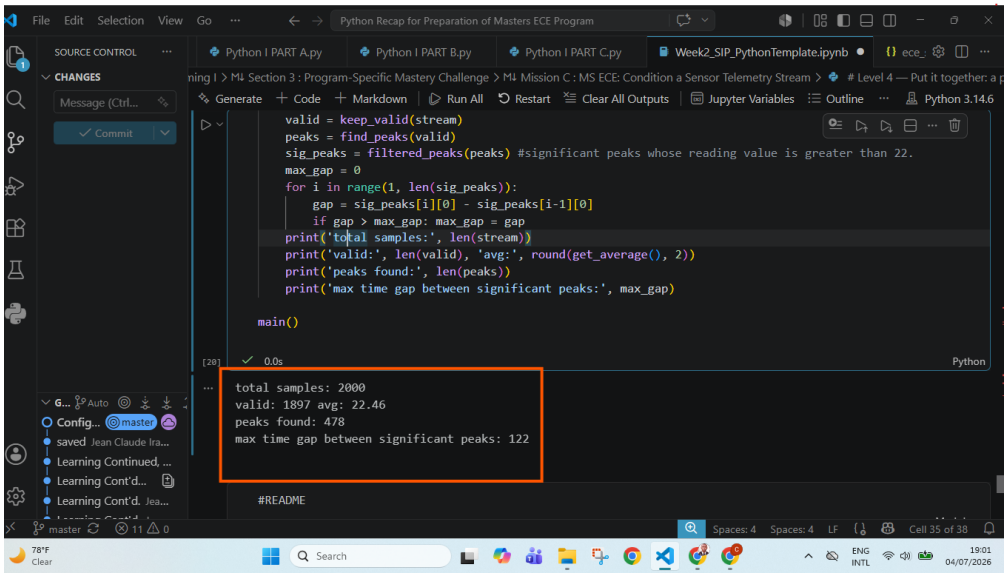

Figure 1. Output of the developed telemetry conditioning system.

The program successfully conditioned the noisy telemetry stream and produced reliable engineering statistics. From the original **2000** telemetry samples, **1987** valid measurements remained after filtering invalid and out-of-range readings. The computed average sensor reading was **22.46**. Furthermore, the peak detection algorithm identified **478** local peaks, while the maximum time gap between consecutive significant peaks was found to be **122** time units. These results demonstrate that the developed pipeline effectively transforms noisy telemetry into meaningful information suitable for engineering analysis.

Conclusion

This assignment demonstrated how Python programming can be used to implement practical engineering solutions while reinforcing Linear Algebra concepts. By decomposing the problem into modular functions, the telemetry conditioning pipeline became easier to develop, test and maintain. The completed system successfully performed data loading, cleaning, aggregation and peak detection,

illustrating the importance of combining mathematical reasoning with structured software design when processing real-world sensor data.

References

1. J. C. IRADUKUNDA, *Python Recap for Preparation of Masters ECE Program*. GitHub Repository.
   https://github.com/iradukunda3546nj/Python-Recap-for-Preparation-of-Masters-ECE-Program
2. W3Schools, *Python Tutorial*.
   https://www.w3schools.com/python/default.asp

Prepared for the Linear Algebra Through Python Programming Mastery Challenge
M.S. Electrical and Computer Engineering
Carnegie Mellon University Africa